MÓDULO 4 — RAG AVANZADO Y TRANSICIÓN A AGENTES · CLASE 8 DE 16

RAG avanzado y transición a Tools

Partimos del límite del RAG fijo: no toda pregunta necesita buscar documentos. Convertimos el RAG construido en una Tool —search_course()— y, desde esa nueva arquitectura, justificamos por qué esa capacidad de búsqueda debe ser confiable: Hybrid Search, RRF y re-ranking. La clase termina con la primera selección entre capacidades y deja preparado el terreno para Function Calling y Agent Loops en C9.

RAG como Tool · Tool descriptions · BM25 · Hybrid Search · RRF · Cross-Encoder · Context Precision · LLM Router

Romper el pipeline

Reconocer cuándo retrieval no es la capacidad correcta para una consulta.

Convertir en Tool

Empaquetar el RAG detrás de una interfaz que el modelo pueda comprender y seleccionar.

Hacerla confiable

Mejorar search_course() con Hybrid, RRF y re-ranking según el tipo de falla.

NÚCLEO OBLIGATORIO

Cambio de arquitectura

RAG deja de ser “la aplicación” y pasa a ser search_course(): una capacidad seleccionable junto con cálculo, APIs o respuesta directa.

PRÁCTICA DE CONSOLIDACIÓN

Confiabilidad de la Tool

Comparar dense, hybrid + RRF e hybrid + reranking reutilizando las métricas de retrieval aprendidas en C6.

AMPLIACIÓN / LUEGO

Evaluación de producción

La evaluación generativa formal y la observabilidad se trabajan más adelante en LLMOps/Producción.

Alcance de esta clase: vas a comprender por qué aparecen las Tools, cómo un modelo puede distinguir capacidades a partir de sus descripciones y por qué el RAG puede convertirse en una Tool confiable. En la Clase 9 vas a avanzar sobre Function Calling, schemas, validación y ejecución controlada.

GUÍA

Pregunta central y conexión curricular

PREGUNTA CENTRAL

¿Qué cambia hoy?

Hasta C7 construimos un pipeline donde toda pregunta terminaba en retrieval. Hoy cuestionamos esa arquitectura: **¿qué pasa cuando la consulta necesita calcular, consultar un dato actual o responder sin buscar?**

OBJETIVOS

Al finalizar deberías poder

- Explicar por qué RAG debe convertirse en una capacidad invocable y no ser siempre el pipeline completo.
- Describir una Tool de forma suficientemente clara para que un LLM pueda distinguir cuándo usarla.
- Diagnosticar fallas de dense retrieval y justificar BM25 + Hybrid + RRF.
- Explicar cuándo un Cross-Encoder mejora el orden de candidatos.
- Comparar alternativas reutilizando Recall/MRR/latencia sin introducir todavía un framework nuevo de evaluación.

LUGAR EN EL CURSO

De C7 a C9

C7: construye RAG E2E. **C8:** convierte ese RAG en `search_course()`, mejora su confiabilidad y presenta un primer **LLM Router** para seleccionar capacidades. **C9:** formaliza Function Calling, schemas, permisos, errores y Agent Loop.

Hilo conductor único

`search_course()` será una Tool del futuro agente. Si esa Tool devuelve evidencia pobre, el agente tomará decisiones con contexto pobre.

> **Hybrid Search, RRF y reranking son ingeniería de confiabilidad de esa capacidad.**

Para profundizar más adelante En Producción/LLMOps vas a trabajar con RAGAS, LLM-as-a-Judge, golden datasets y observabilidad de evaluación. En esta clase usamos Context Precision como una primera intuición para entender por qué importa el orden de la evidencia recuperada.

PARTE 1

El muro del RAG fijo

Retomemos exactamente donde terminó C7

Tenemos un pipeline funcional: **pregunta → retrieval → contexto → LLM → respuesta**. Funciona muy bien cuando la respuesta depende del corpus. El problema aparece cuando convertimos esa arquitectura en una regla universal.

CONSULTA A

Conocimiento del curso

"¿Dónde explicamos MRR?"
Sí necesita retrieval.

CONSULTA B

Cálculo

"Tengo 21 días y usé 8. ¿Cuántos quedan?"
No necesita retrieval.

CONSULTA C

Dato actual

"¿Cuál es hoy el tipo de cambio?"
Necesita una fuente externa vigente.



CAMBIO DE PARADIGMA **Del RAG fijo a un sistema con capacidades.** Retrieval deja de ser el camino obligatorio y pasa a ser una opción entre búsqueda, cálculo, consulta externa o respuesta directa.

Lectura conceptual

Optimizar embeddings o reranking no arregla una consulta que nunca debió ir a RAG. El primer salto hacia agentes no es "más inteligencia": es permitir que el sistema **elija la capacidad adecuada para el problema**.

El nuevo rol del RAG

En lugar de ser toda la aplicación, el pipeline de C5–C7 se encapsula detrás de una interfaz. El consumidor no debería necesitar saber cómo están implementados Qdrant, embeddings, filtros o chunks.

```python
def search_course(query: str) -> list[dict]:
    """Busca evidencia relevante dentro del corpus del curso."""
    return retriever.search(query)
```

✓ **Primer cambio mental:** todo el RAG construido hasta ahora se transforma en `search_course()`. La complejidad sigue adentro; hacia afuera exponemos una capacidad con un propósito claro.

PARTE 2

Tools: capacidades que el modelo puede distinguir

Definición operativa para C8

Una **Tool** es una capacidad delimitada que la aplicación pone a disposición del modelo para realizar una operación concreta. En esta clase todavía no nos importa el protocolo de ejecución: nos importa que cada capacidad tenga **propósito, entrada y resultado comprensibles**.

Conocimiento`search_course(query)`

Cálculo`calculator(expression)`

Dato vigente`external_lookup(query)`

Sin Tool`respond_directly()`

La anatomía conceptual de una Tool

Una Tool no es solamente una función Python. Es un **puente entre dos mundos**: el mundo probabilístico del LLM, que interpreta lenguaje y propone una acción, y el mundo determinístico de la aplicación, donde una operación concreta se ejecuta bajo reglas controladas. Para razonar sobre una Tool sin entrar todavía en Function Calling, conviene separar tres capas.

01 · IMPLEMENTACIÓN

El código subyacente

Es lo que realmente hace el trabajo: consultar Qdrant, ejecutar un cálculo o llamar una API. **El LLM no necesita ver este código**; la aplicación lo mantiene encapsulado.

PYTHON

```python
def search_course(query: str):
    return retriever.search(query)
```

02 · INTERFAZ

Cómo se invoca

Define una superficie estable: **nombre, entradas esperadas y forma de salida**. Permite mejorar la implementación interna sin obligar a cambiar a quien consume la Tool.

```
search_course(query: str) → list[dict]
```

03 · CONTRATO SEMÁNTICO

Qué entiende el modelo

Es la descripción en lenguaje natural que explica **qué resuelve, cuándo conviene usarla y cuándo no**. Es la capa que conecta una intención expresada en lenguaje con una capacidad concreta.

```
“Busca evidencia del curso. No usar para cálculos ni datos actuales.”
```



MODELO MENTAL **Anatomía conceptual de una Tool.** La implementación realiza el trabajo, la interfaz define cómo invocarla y el contrato semántico explica al modelo qué resuelve y cuáles son sus límites.



SELECCIÓN DE CAPACIDADES **Cómo el modelo distingue una Tool.** La solicitud se compara con el contexto y con las descripciones disponibles para proponer la capacidad más adecuada.

¿Cómo sabe el LLM cuál usar?

A nivel conceptual, el modelo compara la solicitud actual con el contexto y con las **descripciones de capacidades disponibles**. Una descripción imprecisa hace más difícil seleccionar bien; una descripción que marca propósito y límites reduce ambigüedad.

Descripción pobre

No delimita intención ni frontera

```python
def search_course(query: str):
    """Busca cosas del curso."""
```

¿Qué significa “cosas”? ¿Cuándo usarla? ¿Sirve para cálculos? ¿Para noticias?

Descripción útil

Explica propósito y límites

```python
def search_course(query: str):
    """Busca evidencia en el corpus del curso.
    Usar para preguntas sobre contenidos internos.
```

```
    No usar para cálculos ni datos externos actuales.
    """
```

La descripción funciona como parte de la interfaz semántica entre lenguaje y capacidad.

De un System Prompt clásico a uno con capacidades

ASISTENTE CLÁSICO

```
Sos un asistente del curso.
Respondé la pregunta usando el contexto disponible.
```

ASISTENTE CON CAPACIDADES · CONCEPTUAL

```
Disponés de:
- search_course: evidencia del curso
- calculator: cálculos
- external_lookup: datos vigentes

Elegí una capacidad solo cuando sea necesaria.
Si ninguna aplica, respondé directamente.
```

⚠ **Descripción ≠ permiso.** Que el modelo entienda una Tool no significa que pueda ejecutarla libremente. Validación, schemas, autorización, timeouts y side effects pertenecen a C9/C11.

PARTE 3

Si search_course() es una Tool, tiene que recuperar bien

Ahora sí aparece RAG avanzado por necesidad

El futuro agente puede elegir correctamente `search_course()` y aun así fallar si la Tool devuelve evidencia pobre. Ese es el problema que resolvemos con Hybrid Search y RRF.

Dense vs. lexical retrieval: dos señales distintas

Dense retrieval

Representa la consulta y los documentos como **vectores densos** y busca cercanía semántica. Es fuerte para paráfrasis, intención y significado, incluso cuando las palabras exactas no coinciden.

Lexical / sparse retrieval

Trabaja principalmente con los **términos presentes en el texto**. Es especialmente útil para códigos, nombres propios, siglas, identificadores y coincidencias exactas.

Dense retrieval no falla siempre por “mal embedding”

Los vectores son fuertes para paráfrasis e intención, pero pueden perder identificadores, códigos, nombres raros o coincidencias exactas. BM25 aporta una señal lexical complementaria: premia coincidencias de términos ponderando rareza, frecuencia y longitud.

```
BM25(d, q) ≈ Σ IDF(term) × función_de_frecuencia(term, d) × normalización_por_longitud(d)
```

La ecuación no es el objetivo. La intuición sí: **la búsqueda lexical protege términos exactos que la representación semántica puede diluir**.

Hybrid Search: dos señales, una lista candidata

Hybrid no significa sumar ingenuamente scores. Similitud vectorial y BM25 viven en escalas distintas. En esta clase usamos **Reciprocal Rank Fusion (RRF)** para combinar posiciones.

VISUAL CONCEPTUAL **Hybrid Search con RRF.** Dense retrieval y BM25 generan rankings independientes; RRF combina sus posiciones sin asumir que los scores de ambas señales son directamente comparables.

Por qué RRF encaja bien acá

El agente no debería preocuparse por esta ingeniería interna. Solo necesita una `search_course()` que entregue candidatos mejores. RRF permite fusionar rankings sin fingir que los scores lexicales y vectoriales significan lo mismo.

```
RRF_score(doc) = Σ 1 / (k + rank_i(doc))
```

🧠 RRF no arregla malos retrievers: si ambas listas contienen evidencia irrelevante, la fusión también será mala. La técnica correcta depende del síntoma.

PARTE 4

Re-ranking: recuperar amplio, leer fino

Bi-Encoder vs. Cross-Encoder: velocidad primero, precisión después

Bi-Encoder

Codifica la query y cada documento **por separado**. Los embeddings del corpus pueden precalcularse e indexarse, por eso permite buscar rápidamente entre muchos documentos.

Cross-Encoder

Recibe **query + documento juntos** y estima directamente la relevancia de ese par. Suele ordenar con más precisión, pero requiere una evaluación nueva para cada candidato y por eso es más costoso.

Principio de dos etapas

Usamos un Bi-Encoder o retriever eficiente para recuperar muchos candidatos y un Cross-Encoder para reordenar solo un conjunto pequeño. Así combinamos escala y precisión.

El siguiente problema: el relevante aparece, pero demasiado abajo

Hybrid puede mejorar el conjunto candidato, pero no garantiza que los mejores chunks queden arriba. Un **Cross-Encoder** recibe el par *(query, chunk)* y evalúa esa relación de manera conjunta, con más precisión pero también con más costo.

¿Qué es el candidate set?

Es el
conjunto inicial de documentos o chunks recuperados antes del reranking
. El reranker puede cambiar su orden, pero no puede recuperar un documento que nunca entró en ese conjunto.

TWO-STAGE RETRIEVAL **Recuperar amplio, reordenar fino.** La primera etapa prioriza recall y velocidad; el Cross-Encoder trabaja sobre pocos candidatos para mejorar el orden final.

> **Regla de diagnóstico**
>
> Si el documento relevante **no entra** al candidate set, debemos mejorar retrieval. Si **entra pero queda mal ordenado**, el reranking es un candidato natural.

Context Precision: una primera intuición

Para entender el valor del reranking alcanza con una idea: no solo importa recuperar evidencia relevante, también importa que esa evidencia aparezca **antes que el ruido** dentro del contexto que verá el modelo.

INTUICIÓN DE EVALUACIÓN **Context Precision y orden del contexto.** No solo importa recuperar evidencia relevante: conviene que aparezca antes que el ruido dentro del contexto que recibirá el modelo.

Más adelante En Producción/LLMOps vas a implementar Context Precision, Faithfulness, datasets de evaluación, tracing y observabilidad para medir el comportamiento del sistema de forma más completa.

PARTE 5

Ingeniería de confiabilidad de search_course()

No buscamos “el RAG más sofisticado”

Comparamos tres implementaciones de la misma Tool: **A) dense**, **B) dense + BM25 + RRF** y **C) hybrid + reranking**. Reutilizamos el conjunto de consultas y las métricas de retrieval conocidas de C6. No abrimos un framework nuevo de evaluación.

Variante de search_course()	Recall@k	MRR	Latencia	Lectura
Dense baseline	—	—	—	Baseline
Hybrid + RRF	—	—	—	¿mejora exact-match?
Hybrid + reranking	—	—	—	¿mejora el orden?

Elegir la técnica por síntoma

> **Códigos, siglas, nombres exactos**
>
> Probar señal lexical / BM25 y luego fusionar con dense.

Paráfrasis o intención semántica

Dense retrieval sigue siendo una señal central.

Relevante dentro del top N pero abajo

Probar Cross-Encoder sobre el conjunto reducido.

Relevante nunca aparece

No culpar al reranker: revisar retrieval, chunking, filtros o corpus.

Criterio de diseño

Una Tool confiable no es la que tiene más componentes, sino la que resuelve sus familias de consultas con calidad suficiente y un costo operativo razonable.

PARTE 6

Semantic Router / LLM Router — decidir antes de ejecutar

Un cambio de rol para el modelo

Hasta ahora usamos el LLM principalmente para **producir una respuesta**. En este patrón le damos primero una tarea más acotada: actuar como **router semántico**. Su responsabilidad inicial no es resolver la consulta, sino **interpretar la intención y derivarla hacia la capacidad adecuada**.

Patrón: Semantic Router

Entrada:

consulta del usuario + descripciones de capacidades.

Salida:

una decisión de ruteo. En C8 usamos un LLM como clasificador porque reutiliza Structured Outputs de C2; el patrón también puede implementarse con reglas, clasificadores o embeddings cuando el problema lo permita.

CONOCIMIENTO INTERNO

“¿Qué es RRF?”

→ `search_course()`

OPERACIÓN EXACTA

“21 – 8”

→ `calculator()`

SIN HERRAMIENTA

“Explicame esta frase”

→ `direct_answer` si el contexto alcanza

El router no responde: clasifica la intención

El primer llamado al modelo puede devolver una estructura mínima, por ejemplo `{"route": "search_course"}`. La aplicación usa esa decisión para elegir el siguiente camino. Esto evita enviar todas las consultas al mismo pipeline y **reduce respuestas improvisadas cuando existe una capacidad más precisa**.

Pipeline fijo

Una sola ruta para todo

```
consulta → RAG → respuesta
```

Una cuenta matemática o una pregunta sin necesidad de documentos también atraviesan retrieval, aunque no corresponda.

Semantic Router

Primero decide el tipo de trabajo

```
consulta → router → capacidad adecuada
```

La arquitectura especializa el tratamiento: conocimiento, cálculo, datos externos o respuesta directa.

Router ≠ Agent

Un router toma una decisión acotada:
“¿qué camino corresponde?”
. Un agente, en cambio, puede elegir una Tool, ejecutarla, observar el resultado y
volver a decidir
. Ese ciclo de decisión → acción → observación es el salto que vamos a formalizar en la próxima clase.

⚠ **Routing no elimina las alucinaciones.** Reduce el riesgo de usar una capacidad inadecuada, pero el router puede clasificar mal y una Tool puede devolver datos pobres. Por eso seguimos necesitando validación, observabilidad y evaluación.

JSON · STRUCTURED OUTPUT

```
{
  "route": "calculator"
}
```

En C8 esta salida es solamente una **decisión de ruteo**. La aplicación decide qué función invocar mediante código explícito. En C9 reemplazamos este mecanismo artesanal por Tool/Function Calling con argumentos estructurados, validación y ejecución controlada.

PYTHON CONCEPTUAL

```
decision = route(query)

if decision["route"] == "search_course":
    result = search_course(query)
elif decision["route"] == "calculator":
    result = calculator(query)
else:
    result = direct_answer(query)
```

TRANSICIÓN A AGENTES **El router decide el camino; C9 formaliza la ejecución.** En esta clase el modelo clasifica la intención y selecciona una capacidad; luego veremos cómo declarar Tools, validar argumentos, ejecutarlas y observar resultados.

Pregunta de salida

Si hoy el router solo devuelve una etiqueta como `search_course` o `calculator`, ¿cómo hacemos para que mañana el modelo pueda declarar una Tool, proponer argumentos válidos, ejecutarla de forma controlada y observar el resultado para decidir otra vez?

NOTEBOOK

Notebook guiada — hacer confiable search_course()

PRÁCTICA GUIADA

Una misma interfaz, tres estrategias internas

La notebook compara variantes de retrieval manteniendo estable la interfaz `search_course(query)`. La idea es que el consumidor no cambie mientras mejoramos la implementación interna.

CHECKPOINT 1

Dense baseline

Recuperar el estado de C7 y medir el retriever sin cambios usando las métricas ya conocidas de C6.

CHECKPOINT 2

Hybrid + RRF

Agregar señal lexical, observar queries exactas y fusionar rankings conservando trazabilidad de fuentes.

CHECKPOINT 3

Reranking

Aplicar Cross-Encoder sobre pocos candidatos y verificar si mejora el orden a un costo razonable.

Alcance de la notebook

La notebook se concentra en comparar retrieval y ranking. La evaluación generativa completa se trabaja más adelante junto con tracing y observabilidad.

Cierre técnico de la notebook

PYTHON

```python
def search_course(query: str) -> list[dict]:
    """Busca evidencia en el corpus del curso.

    Usar para preguntas cuyo contenido depende del material interno.
```

```
    No usar para cálculos ni información externa en tiempo real.
    """
    return rag_service.search(query)
```

DEFINITION OF DONE

- [ ] Dense, Hybrid+RRF y Rerank se compararon sobre las mismas consultas
- [ ] La comparación reutiliza Recall/MRR y latencia sin abrir un framework nuevo
- [ ] `search_course()` mantiene una interfaz estable
- [ ] La descripción de la Tool explicita propósito y límites

ACTIVIDAD

RAG Tournament — de diagnóstico a Tool Design

DINÁMICA GRUPAL

La meta no es agregar componentes: es diseñar una capacidad que el agente pueda usar bien

Cada equipo recibe consultas problemáticas, diagnostica la falla, mejora `search_course()` y después debe explicar al futuro agente cuándo esa Tool debería —y cuándo no debería— utilizarse.

PASO 1 · DIAGNÓSTICO

¿La pregunta necesita RAG?

- search_course
- calculator
- external_lookup
- respuesta directa

PASO 2 · RETRIEVAL

Si necesita RAG, ¿qué falla?

- exact-match → lexical
- candidate set → hybrid/RRF
- orden → reranker
- corpus/chunking → no maquillar

PASO 3 · INTERFAZ

Escriban la descripción

Redacten el docstring / descripción que verá el modelo: propósito, cuándo usarla, cuándo no y qué devuelve.

Restricción

No se acepta “usar Hybrid + Rerank porque es mejor”. Cada componente debe responder a una falla observada y cada Tool debe tener límites explícitos.

Canvas de entrega

Consulta	Tool elegida	Señal que justificó la elección	Falla / mejora de retrieval	Descripción de la Tool
Consulta 1	—	—	—	—
Consulta 2	—	—	—	—
Consulta 3	—	—	—	—

Ejemplo de una buena descripción

TEXT

```
search_course(query)
Busca evidencia dentro del material oficial del curso.
Usar cuando la respuesta depende de conceptos, definiciones o ejemplos del corpus.
No usar para cálculos ni para datos externos que cambian en tiempo real.
Devuelve chunks relevantes con fuente y metadata.
```

RECURSOS

Para profundizar

RETRIEVAL

BM25, Hybrid Search y RRF

Profundizar las diferencias entre señal lexical y semántica y revisar la implementación utilizada en Qdrant.

BM25 · Hybrid · RRF

RANKING

Cross-Encoders

Revisar por qué un modelo que ve query y chunk juntos puede mejorar el orden de un conjunto candidato pequeño.

Reranking · Cross-Encoder

PRÓXIMA CLASE

Tool Engineering y Function Calling

La descripción semántica introducida hoy se convierte en C9 en un contrato formal con schemas, validación, permisos y ejecución controlada.

Tools · Function Calling

MÁS ADELANTE

Evaluación generativa y observabilidad

Datasets formales, jueces, tracing y frameworks de evaluación se trabajan juntos en Producción/LLMOps.

LLMOps · Evaluation · Observability

CIERRE

Qué tiene que quedar de la Clase 8

01

RAG no es toda la aplicación

Es una capacidad apropiada cuando la respuesta depende de evidencia del corpus.

02

Una Tool necesita una buena descripción

El modelo necesita entender propósito y límites para poder distinguirla de otras capacidades.

03

Hybrid protege señales distintas

Dense captura semántica; BM25 protege coincidencias lexicales; RRF fusiona rankings.

04

Rerank mejora el orden

Cuando el relevante ya está entre los candidatos, un Cross-Encoder puede leer fino sobre pocos chunks.

05

La calidad de la Tool importa

Un agente que selecciona correctamente una Tool defectuosa seguirá tomando decisiones con evidencia pobre.

06

C9 formaliza la acción

Function Calling, schemas, permisos, errores y Agent Loop convierten la selección conceptual en ejecución controlada.

Próxima clase

Clase 9 — Agents + Tools: del “qué capacidad usar” al “cómo declararla, ejecutarla y observarla”
